

Mejores Prácticas y Guía de Documentación

1. Cómo Documentar el Proyecto

1.1 Estructura de Documentos

```
docs/
├── arquitectura/
│   ├── modelo_datos.md
│   └── flujo_sistema.md
├── api/
│   ├── endpoints.md
│   └── autenticacion.md
├── desarrollo/
│   ├── setup_local.md
│   └── deployment.md
└── usuario/
    ├── manual.md
    └── faq.md
```

1.2 README del Proyecto

```
# English Learning Platform

## Descripción
Plataforma web para aprendizaje de inglés...

## Tech Stack
- Backend: Python Flask 3.0
- Frontend: HTML5, Bootstrap 5
- DB: PostgreSQL

## Quick Start
```bash
pip install -r requirements.txt
cp .env.example .env
flask run
```

## Estructura

```
app/
├── routes/ # Blueprints
├── services/ # Lógica de negocio
├── models/ # Modelos SQLAlchemy
└── templates/ # Jinja2
```

# Contributing

Ver CONTRIBUTING.md

## Licencia

MIT

```

2. Cómo Documentar Funciones y Clases Python

2.1 Docstrings según Tipo

Clase:
```python
class User(UserMixin, db.Model):
    """
    Modelo de usuario principal.

    Attributes:
        id (int): Identificador único.
        username (str): Nombre de usuario único.
        email (str): Correo electrónico único.
        subscription_type (str): Tipo de suscripción.

    Relationships:
        progress: Relación uno a muchos con UserProgress.
        badges_earned: Relación muchos a muchos con Badge.

    Example:
        >>> user = User(username='john', email='john@example.com')
        >>> user.set_password('password123')
        >>> db.session.add(user)
    """
```

Función/Método:

```
def has_access_to_scenario(self, scenario_id):
    """
    Verifica si el usuario puede acceder a un escenario.

    Args:
        scenario_id (int): ID del escenario a verificar.

    Returns:
        bool: True si tiene acceso, False en caso contrario.
```

```

Raises:
    ValueError: Si el escenario_id es inválido.

Note:
    Los administradores siempre tienen acceso.
    Los usuarios premium tienen acceso total.
"""

```

Función simple:

```

def calculate_streak(days_active):
    """Calcula la racha basada en días activos consecutivos."""
    return min(days_active, 365) # Max 365 días

```

2.2 Tipos con Type Hints

```

from typing import Optional, List, Dict, Any

def get_user_progress(user_id: int) -> Dict[str, Any]:
    """Obtiene el progreso del usuario."""
    ...

def update_user(
    user_id: int,
    username: Optional[str] = None,
    email: Optional[str] = None
) -> User:
    """Actualiza datos del usuario."""
    ...

def list_units(page: int = 1, per_page: int = 10) -> List[Unit]:
    """Lista unidades con paginación."""
    ...

```

3. Cómo Documentar Endpoints

3.1 Formato por Endpoint

```

### GET /api/v1/units

Obtiene la lista de unidades disponibles.

**Autenticación**: Requiere JWT o sesión activa

**Parámetros Query**:
| Param | Tipo | Requerido | Descripción |

```

```
|-----|-----|-----|-----|
| page | int | No | Página (default: 1) |
| per_page | int | No | Items por página (default: 10) |

**Respuesta 200**:
```json
{
 "units": [
 {
 "id": 1,
 "title": "Unit 1: Basics",
 "description": "Introduction to English",
 "completed": false
 }
],
 "total": 20,
 "page": 1,
 "per_page": 10
}
```

### Errores:

- 401: No autenticado
- 500: Error del servidor

### ### 3.2 Documentación en Código

```
```python
@units_bp.route('/<int:unit_id>', methods=['GET'])
@login_required
def get_unit(unit_id):
    """
    Obtiene los detalles de una unidad específica.

    Args:
        unit_id (int): ID de la unidad.

    Returns:
        200: JSON con datos de la unidad.
        404: Unidad no encontrada.
    """
    unit = Unit.query.get_or_404(unit_id)
    return jsonify({
        'id': unit.id,
        'title': unit.title,
        'description': unit.description,
        'topics': [t.title for t in unit.topics]
    })
```

4. Cómo Documentar Templates Jinja2

4.1 Estructura de Template

```
<!--
Template: dashboard.html

Descripción: Panel principal del usuario tras login.

Bloques:
- content: Contenido principal
- extra_css: CSS adicional
- extra_js: JS adicional

Variables esperadas:
- current_user: Usuario logueado
- stats: Estadísticas del usuario
- recent_activities: Lista de actividades recientes

Dependencias:
- base.html (hereda)
- components/cards.css
-->
{% extends "base.html" %}

{% block content %}
<div class="row">
  <div class="col-md-8">
    <!-- Contenido principal -->
  </div>
  <div class="col-md-4">
    <!-- Sidebar -->
  </div>
</div>
{% endblock %}
```

4.2 Macros Reutilizables

```
{% macro render_card(title, content, icon='fas fa-book') %}
<div class="card">
  <div class="card-body">
    <h5 class="card-title">
      <i class="{{ icon }}"></i> {{ title }}
    </h5>
    <p class="card-text">{{ content }}</p>
  </div>
</div>
{% endmacro %}

<!-- Uso -->
```

```
{{ render_card('Gramática', 'Aprende las reglas gramaticales', 'fas fa-book-open') }}
```

5. Cómo Documentar Base de Datos

5.1 Modelo Individual

Tabla: users

Descripción

Almacena información de los usuarios registrados en la plataforma.

Campos

Campo	Tipo	Null	Default	Descripción
id	INT	No	AUTO_INCREMENT	PK
username	VARCHAR(80)	No	-	Unique, indexed
email	VARCHAR(120)	No	-	Unique, indexed
password_hash	VARCHAR(255)	No	-	Bcrypt hash
created_at	DATETIME	No	CURRENT_TIMESTAMP	

Índices

- PRIMARY KEY (id)
- UNIQUE INDEX idx_username (username)
- UNIQUE INDEX idx_email (email)

Relaciones

- 1:N → user_progress
- 1:N → subscriptions
- N:N → badges (via user_badges)
- N:N → thematic_scenarios (via user_unlocked_scenarios)

Notas

- password_hash usa Werkzeug (bcrypt)
- is_admin determina acceso al panel admin

6. Cómo Mantener Escalabilidad

6.1 Estructura de Rutas (Refactorizar)

Antes (problema):

```
app/routes/
├─ grammar.py      # 1822 líneas - MONOLÍTICO
├─ exams.py        # Muy grande
└─ ...
```

Después (recomendado):

```
app/routes/
├── grammar/
│   ├── __init__.py      # Blueprint registration
│   ├── rules.py         # /grammar/rules
│   ├── exercises.py     # /grammar/exercises
│   └── api.py           # /grammar/api
├── exams/
│   ├── __init__.py
│   ├── list.py
│   ├── take.py
│   └── results.py
└── ...
```

6.2 Patrón Repository

```
# app/repositories/user_repository.py
class UserRepository:
    """Repositorio para operaciones de usuario."""

    @staticmethod
    def get_by_id(user_id: int) -> Optional[User]:
        return User.query.get(user_id)

    @staticmethod
    def get_by_email(email: str) -> Optional[User]:
        return User.query.filter_by(email=email).first()

    @staticmethod
    def create(username: str, email: str, password: str) -> User:
        user = User(username=username, email=email)
        user.set_password(password)
        db.session.add(user)
        db.session.commit()
        return user

    @staticmethod
    def update_last_login(user: User) -> None:
        user.last_login_date = datetime.date.today()
        db.session.commit()
```

6.3 Patrón Service Layer

```
# app/services/interfaces/i_user_service.py
from abc import ABC, abstractmethod

class IUserService(ABC):
```

```

@abstractmethod
def register(self, username: str, email: str, password: str) -> User:
    pass

@abstractmethod
def authenticate(self, email: str, password: str) -> Optional[User]:
    pass

@abstractmethod
def get_profile(self, user_id: int) -> Dict[str, Any]:
    pass

# app/services/user_service.py
class UserService(IUserService):
    def __init__(self, user_repo: UserRepository):
        self._user_repo = user_repo

    def register(self, username: str, email: str, password: str) -> User:
        # Validaciones
        # Log de auditoría
        return self._user_repo.create(username, email, password)

```

6.4 Separación de Responsabilidades

```

Routes      → Controladores (HTTP)
Services    → Lógica de negocio
Repositories → Acceso a datos
Models      → Representación datos

```

7. Checklist de Documentación

Antes de Commit

- ☐ Docstrings en funciones nuevas
- ☐ Type hints donde aplica
- ☐ Comments en lógica compleja
- ☐ README actualizado si hay cambios en setup

Revisión de Código

- ☐ Nombres descriptivos de variables
- ☐ Funciones pequeñas (< 50 líneas)
- ☐ Una responsabilidad por función
- ☐ Sin código comentado (o marcar TODO)

Proyecto

- ☐ Estructura consistente

- ☐ Convenciones de nombres seguidas
 - ☐ Dependencias documentadas en requirements.txt
-

8. Convenciones del Proyecto

Naming

- **Archivos Python:** snake_case (`user_service.py`)
- **Clases:** PascalCase (`UserService`)
- **Funciones:** snake_case (`get_user_by_id`)
- **Constantes:** UPPER_SNAKE_CASE (`MAX_RETRY`)
- **Templates HTML:** snake_case (`user_profile.html`)
- **Rutas URL:** kebab-case (`/user-profile`)

Imports

```
# Standard library
import os
from datetime import datetime

# Third party
from flask import jsonify
from flask_login import login_required

# Local
from app.models import User
from app.extensions import db
```

Config

```
# config.py - todo en mayúsculas
SECRET_KEY = os.environ.get('SECRET_KEY')
DATABASE_URL = os.environ.get('DATABASE_URL')
```